4.d Classes - Interfaces

Understanding Kotlin Language Fundamentals

Contents

1 **Interfaces Overview**
Definition, declaration syntax, implementing interfaces

2 **Properties in Interfaces**
Abstract properties, default accessors, interface inheritance

3 **Overriding Conflicts**
Multiple inheritance resolution with super<T> syntax

4 **Functional (SAM) Interfaces**
SAM conversion, lambda expressions, type aliases comparison

What Are Interfaces?

Interfaces in Kotlin can contain declarations of abstract methods, as well as method implementations. Unlike abstract classes, interfaces cannot store state. They are declared using the **interface** keyword.

Key characteristics of interfaces:

Abstract Methods

Methods without a body are abstract by default. Implementing classes must override them.

Default Implementations

Methods can have a body, providing a default implementation that implementing classes may inherit.

No State Storage

Properties can be declared but cannot have backing fields. Accessors cannot reference them.

Multiple Inheritance

A class can implement multiple interfaces. But only single-class inheritance is allowed.

Implementing Interfaces

A class implements an interface using a colon **:** after the class name. It must override all abstract members *.

Declaring an Interface

```
interface MyInterface {
    fun bar()           // abstract member
    fun foo() {
        println("foo")
    }
}
```

Implementing the Interface

```
class Child : MyInterface {
    override fun bar() {
        println("bar")
    }
}

val child = Child()
child.bar() // Output: bar
child.foo() // Output: foo    // foo() is inherited
```

💡 **Note:** Only abstract members (without a body) must be overridden *. Methods with default implementations are inherited automatically.

* For non-abstract classes

Properties in Interfaces

Interfaces can declare properties. These can be abstract or provide default accessor implementations. Properties in interfaces **cannot have backing fields**.

Interface with Properties

```
interface MyInterface {
    val prop: Int  // abstract

    val propWithImpl: String
        get() = "foo"
}

fun testInterfaces1() {
    val obj = object : MyInterface {  // anonymous class
        override val prop: Int
            get() = 42
    }

    println(obj.prop)         // Output: 42
    println(obj.propWithImpl) // Output: foo
}
```

Implementing Class

```
class Child : MyInterface {
    override val prop: Int = 29
}

val child = Child()
println(child.prop)         // Output: 29
println(child.propWithImpl) // Output: foo

// Only abstract property
// 'prop' must be overridden.
// 'propWithImpl' is inherited.
```

Interface Inheritance

An interface can derive from other interfaces. It can both provide implementations for their members and declare new functions and properties. Classes implementing such an interface only need to define the missing implementations.

Example: Deriving Interfaces

```
interface Named {
    val name: String
}

interface Person : Named {
    val firstName: String
    val lastName: String
    override val name: String
        get() = "$firstName $lastName"
}

data class Employee (
    override val firstName: String,
    override val lastName: String
) : Person  // 'name' is inherited!


val emp1 = Employee("John", "Doe")
println(emp1.name) // John Doe
```

Resolving Overriding Conflicts

When a class inherits from multiple interfaces that define the same method, the class must override it and specify how it should behave. Use **super<InterfaceName>** to call a specific parent implementation.

Two Interfaces

```
interface A {
    fun foo() { println("A") }
    fun bar()
}

interface B {
    fun foo() { println("B") }
    fun bar() { println("bar") }
}
```

Resolving with super<T>

```
class D : A, B {
    override fun foo() {
        super<A>.foo()
        super<B>.foo()
    }
    override fun bar() {
        super.bar()
    }
}

val d = D()
d.foo()
d.bar()
```

```
Output:
A
B
bar
```

💡 **Rule:** If you inherit multiple implementations of the same method, you must override it — even for methods with only one inherited implementation (like bar()).

Functional (SAM) Interfaces

An interface with **only one abstract method** is called a functional interface, or **Single Abstract Method (SAM) interface**. It can have multiple non-abstract members but only one abstract member. Declared using the **fun interface** keywords.

Key characteristics:

Declaration

Use the **fun** modifier before the interface keyword to declare a functional interface (SAM).

SAM Conversion

Use a lambda expression instead of manually creating a class that implements the interface. Reduces boilerplate code.

Flexibility

Can have multiple non-abstract members. Can also implement and extend other interfaces.

SAM Conversions in Practice

❌ Without SAM Conversion

```
fun interface IntPredicate {
    fun accept(i: Int): Boolean
}

// Verbose: creating an anonymous object
val isEven = object : IntPredicate {
    override fun accept(i: Int) = i % 2 == 0
}

println(isEven.accept(7))  // Output: false
```

✅ With SAM Conversion

```
// Concise, with lambda
val isEven2 = IntPredicate {
    it % 2 == 0
}

println(isEven2.accept(7))  // Output: false
```

Functional Interfaces vs Type Aliases

Functional Interfaces

- Create a new type
- Can have multiple non-abstract members
- Can extend other interfaces
- More flexible but slight runtime overhead

Type Aliases

- Just a name for an existing type
- Can have only one member
- No extension capabilities
- Simpler, no runtime cost

Practice Exercises

Solved Exercise 1

Create an interface **Shape** with an abstract method **area(): Double**. Implement it in a **Circle** class.

✅ Solution

```
interface Shape {
    fun area(): Double
}

class Circle(val radius: Double) : Shape {
    override fun area(): Double =
        Math.PI * radius * radius
}

fun main() {
    val circle = Circle(5.0)
    println("Area: ${circle.area()}")
}  // Output: Area: 78.539...
```

Solved Exercise 2

Declare two interfaces **ShortGreeter** and **FullGreeter**, each defining a method fun greet(text: String). The **ShortGreeter** should print the received text truncated to **10 characters** followed by "...", while the **FullGreeter** should print the **entire text**. Then declare a class **SmartGreeter** that implements both interfaces and overrides the method greet(text: String): if the text has **10 characters or fewer**, it should call the **full greeting**, otherwise it should call the **short greeting** using the text truncated to **10 characters**. Finally, write a small main function that creates a SmartGreeter instance and tests it with strings of different lengths.

✅ Solution

```kotlin
interface ShortGreeter {
    fun greet(text: String) {
        println(text.take(10) + "...")
    }
}

interface FullGreeter {
    fun greet(text: String) {
        println(text)
    }
}

class SmartGreeter : ShortGreeter, FullGreeter {
    override fun greet(text: String) {
        if (text.length <= 10)
            super<FullGreeter>.greet(text)
        else
            super<ShortGreeter>.greet(text)
    }
}
```

```kotlin
fun testInterfaces3() {
    val g = SmartGreeter()

    g.greet("Hello")
    g.greet("Hello Kotlin Programming")
}
```

```
Output:
Hello
Hello Kotl...
```

Unsolved Exercise 1

Define an interface **Describable** with an abstract property **description: String** and a default method **describe()** that prints it. Create a class **Book** that implements it.

📋 Requirements

Interface

- Abstract property
- Default method

Class Book

- Override property
- Use constructor param

Expected Output

```
Description: Kotlin
in Action
```

💡 Hints

- Interface properties are declared with **val** (no backing field allowed)
- Override the property in the constructor: `class Book(override val description: String)`
- Use string templates with `$description` in the default method

Unsolved Exercise 2

Create a functional (SAM) interface **StringTransformer** with a single method **transform(s: String): String**. Use a SAM conversion (lambda) to create an instance that converts text to uppercase, and print the result.

📋 Requirements

SAM Interface

- Use fun interface
- Single abstract method

Lambda Usage

- SAM conversion
- it.uppercase()

Expected Output

```
HELLO KOTLIN
```

💡 Hints

- Declare the interface with `fun interface StringTransformer { ... }`
- Create instance via SAM: `val upper = StringTransformer { it.uppercase() }`
- Call with `println(upper.transform("hello kotlin"))`